

근위흉추만곡의 척추경 폭(width)의 베イズ 통계를 이용한 분석

2018년의 빈도주의 통계를 이용한 분석(박건보 교수 논문)을 베イズ 통계를 이용하여 다시 분석한다.

기왕에 작성한 논문을 베이지안 관점에서 해석

베イズ 통계학을 **자신의 것으로 만든다는 것**은 단순히 새로운 분석 도구를 갖는 것을 넘어, 기존 빈도주의 관점(p 값 중심)에서 작성된 연구들을 전혀 새로운 시각으로 재해석할 수 있음을 의미한다. 이는 기존 데이터를 활용해 또 다른 논문을 쓸 수 있다는 실무적인 이점뿐만 아니라, 데이터 뒤에 숨겨진 불확실성을 더 깊이 이해하는 통찰을 제공한다.

독자 여러분들도 빈도주의 통계로 작성했던 기존 논문을 베イズ 관점에서 재해석해 볼 것을 진심으로 권한다. 이는 단순히 관점의 변화를 넘어, 여러분의 소중한 데이터로부터 전혀 새로운 또 하나의 논문이 탄생하는 계기가 될 것이다.

필자는 이러한 시도의 일환으로, 2018년에 발표했던 ‘척추경 넓이(Pedicle Width)’에 관한 논문을 베이지안 관점에서 재분석하였다[Lee et al., 2018].

0.1 코드 분석

두 개의 파일이 같은 폴더 내에 있는 경우 작업 중 간혹 작업 폴더(working directory)가 달라져 “file not found” 오류가 발생한다. 이 오류를 막기 위하여 RProj & here 워크플로우를 사용한다.

이를 위하여 프로젝트 폴더 `ais_scoliosis_analysis`를 먼저 만들었다. 프로젝트 폴더 내에 레시피 코드, Stan 코드, 메인 코드의 세 가지 코드와 RProj 파일(`ais_scoliosis_analysis`) 등이 있다. 파일을 열 때는 RProj 파일을 통하여 여는게 여러 모로 편리하다.

- 레시피 코드: `data.R`
- Stan 코드: `simple_model.stan`
- 메인 코드: `main_analysis.R`

0.1.1 레시피 코드

레시피 코드인 data.R 파일은 다음과 같다.

```
# data.R (척추경 넓이 전용 레시피)
library(readxl)
library(tidyverse)
library(here)

# 1. 엑셀 파일 로드
raw_df <- read_excel(here("kbpark_modify.xlsm"), skip = 1)

# data.R 2번 섹션 최종 수정
clean_df <- raw_df %>%
  select(ID, 33:66) %>%
  rename_with(~str_remove(., "\\\\.\\.\\.\\.\\..*"), -ID) %>%
  pivot_longer(cols = -ID, names_to = "key", values_to = "width") %>%
  mutate(
    Level = str_extract(key, "[TL]\\d+"),
    Side_Label = ifelse(str_detect(key, "Lt"), "Left", "Right")
  ) %>%
  filter(!is.na(width), width > 0) %>%
  # --- 여기서 T1~T6만 골라내는 '여과지'를 추가합니다 ---
  filter(Level %in% c("T1", "T2", "T3", "T4", "T5", "T6")) %>%
  # -----
  mutate(level_idx = as.integer(factor(Level, levels =
    c("T1", "T2", "T3", "T4", "T5", "T6"))))

# 3. Stan 데이터 리스트 (level_idx를 사용합니다)
stan_data <- list(
  N = nrow(clean_df),
  width = as.numeric(clean_df$width),
  vertebra_level_numeric = clean_df$level_idx,
  side_numeric = ifelse(clean_df$Side_Label == "Left", 1, 2)
)
```

코드를 하나씩 설명한다.

- `raw_df <- read_excel(here("kbpark_modify.xlsm"), skip = 1)`는 excel file을 읽어서 `raw_df`로 만드는 것이다.

- * `names_to = "key"`: LtT1~LtT9 같은 열 이름들은 key라는 새로운 열로 모으고,
 - * `values_to = "width"`: 그 안에 들어있던 실제 측정값(척추경 넓이)은 width라는 열로 재배치하라는 뜻이다.
- ▷ `mutate()` 함수는 데이터프레임에서 새로운 열(column)을 만든다.
- 여기서는 Level, Side_Label의 두 열을 만드는데,
- * `Level=str_extract(key, "[TL]\\d+")` 코드에서 [TL]은 T 혹은 L로 시작하는 글자를 찾으라는 뜻이고, \\d+는 그 뒤에 붙은 숫자(digit)를 모두 가져오라는 뜻이다. 그 결과 LtT1에서는 T1을, RtL5에서는 L5를 쏙 뽑아내어 Level이라는 열에 담는다.
 - * `Side_Label=ifelse(str_detect(key, "Lt"), "Left", "Right")`는 key 열의 글자 중에 Lt라는 글자가 포함되어 있는지 찾아본다. 만약 Lt가 발견되면(TRUE) Left라고 쓰고, 발견되지 않으면(FALSE) Right라고 쓰라는 명령어이다.
- ▷ `filter()` 함수는 수많은 데이터 중에서 우리가 원하는 조건을 만족하는 진짜 데이터만 남기고 나머지는 걸러내는 거름망 역할을 한다. `filter(A, B)`는 A & B의 두 조건을 동시에 만족해야 한다는 뜻이다.
- * `filter(!is.na(width), width > 0)` 코드는 width 열의 값이 NA(Not Available)인지 물어보는 함수로, !(반대 기호)가 붙어서, ‘값이 비어있지 않은 데이터만 통과시켜라’는 뜻이다. 엑셀에서 측정값을 적지 않고 비워둔 행들을 걸러내는 작업이다. 또한 `width>0`에 따라 측정값이 0보다 큰 경우만 남기라는 뜻이다.
 - * `filter(Level %in% c("T1", "T2", "T3", "T4", "T5", "T6"))%>%`: Level 내의 글자가 c()라는 바구니 안에 포함되어 있는 것만 취하라는 뜻이다.
- ▷ `mutate(level_idx = as.integer(factor(Level, levels = c("T1", "T2", "T3", "T4", "T5", "T6"))))`

이 코드의 의미는 다음과 같다.

- * `factor(Level, levels = c("T1", "T2", "T3", "T4", "T5", "T6"))`는 “이 글자들은 그냥 글자가 아니라 순서와 계급이 있는 집단이다”라고 선언하는 함수이다.
- * `as.integer(...)`는 그 순서를 정수로 바꾸라는 뜻이다.
- * 변환 결과는 다음과 같다.

표 1: 변환 결과 예시

ID	Level (글자)	level_idx (숫자)
P01	T1	1
P01	T2	2
⋮	⋮	⋮
P01	T6	6

여기서 P01은 1번 환자를 나타낸다.

* 결국 이 코드는 해부학적 용어(T1~ T6)를 컴퓨터가 이해할 수 있는 숫자 언어(1 ~ 6)로 번역한 것이다.

- 결국 연결 연산자 %>%로 길게 연결된 위의 코드에 의하여 생성된 clean_df는 다음과 같다.

```
> clean_df
# A tibble: 2,020 × 6
      ID key   width Level Side_Label level_idx
  <dbl> <chr> <dbl> <chr> <chr>      <int>
1    44 LtT1  4.41 T1    Left         1
2    44 LtT2  3.23 T2    Left         2
3    44 LtT3  2.31 T3    Left         3
4    44 LtT4  2.58 T4    Left         4
5    44 LtT5  2.28 T5    Left         5
6    44 LtT6  2.05 T6    Left         6
7    44 RtT1  4.57 T1    Right        1
8    44 RtT2  3.34 T2    Right        2
9    44 RtT3  3.11 T3    Right        3
10   44 RtT4  3.39 T4    Right        4
# i 2,010 more rows
# i Use `print(n = ...)` to see more rows
```

- Stan file에 공급할data (stan_data)도 여기서 정의한다.

```
stan_data <- list(
  N = nrow(clean_df),
  width = as.numeric(clean_df$width),
  vertebra_level_numeric = clean_df$level_idx,
  side_numeric = ifelse(clean_df$Side_Label == "Left", 1, 2)
)
```

▷ N = nrow(clean_df)는 데이터 프레임 clean_df의 전체 행의 수로 2020이다.

▷ side_numeric = ifelse(clean_df\$Side_Label == "Left", 1, 2)에서 left는 1로, right는 2로 한다.

0.1.2 Stan 코드

Stan 파일명은 simple_model.stan으로, 다음과 같다.

```
// This model estimates a mean pedicle width difference
//                               for each vertebral level (T1-T6).

data {
  int<lower=0> N;
```

```

array[N] int<lower=1, upper=6> vertebra_level_numeric;
array[N] int<lower=1, upper=2> side_numeric; // 1: Left, 2: Right
array[N] real width; // 척추경의 절대 넓이 (mm)
}

parameters {
  // 6개 레벨 x 2개 사이드 = 총 12개의 평균값을 추정한다.
  matrix[6, 2] mu;
  real<lower=0> sigma;
}

model {
  to_vector(mu) ~ normal(5, 3);
  sigma ~ exponential(1);

  for (i in 1:N) {
    width[i] ~ normal(mu[vertebra_level_numeric[i], side_numeric[i]], sigma);
  }
}

generated quantities {
  // 1. 모든 레벨(T1-T6)의 좌우 평균 차이 계산
  matrix[6, 2] mu_prob_narrow;
  for (k in 1:6) {
    for (s in 1:2) {
      // 각 레벨의 각 측면 평균이 2mm 미만일 확률 (지표 변수)
      // 3mm는 나사가 들어갈 수 있는 최소한의 안전 마진 예시입니다.
      mu_prob_narrow[k, s] = (mu[k, s] < 2.0);
    }
  }

  // 2. 전체 데이터에 대한 사후 예측 (모델 검증용)
  array[N] real y_rep;
  for (n in 1:N) {
    y_rep[n] = normal_rng(mu[vertebra_level_numeric[n], side_numeric[n]], sigma);
  }
}

```

stan 파일의 코드를 살펴 보자.

- data 블록에서는 분석에 필요한 데이터의 구조를 선언한다. 여기서는 `vertebra_level_numeric`과 `side_numeric`의 두 가지 변수가 정의되어, 각 데이터 포인트의 해부학적 위치 정보를 담게 된다.
- parameters 블록에서 정의한 모수(parameter)는 `mu`와 `sigma`의 둘이다. `mu`는 척추경의 넓이를 나타내며, 6개의 level 정보와 2개의 side 정보를 통하여 등 총 12개의 척추경 넓이의 평균값을 추정한다. `sigma`는 그 표준편차이다.
- model 블록의 `to_vector`와 `sigma`는 사전분포(prior distribution)를 나타낸다.
 - ▷ 사전분포는 데이터를 관찰하기 전, 우리가 모델의 모수(parameter)에 대해 가지고 있는 배경 지식이나 가정을 나타낸다.
 - ▷ `mu`는 행렬 형태를 취하는데, Stan은 행렬 자체를 정규분포에 할당하는 것을 허용하지 않는다.
`to_vector(mu)` 함수는 행렬(matrix) 형태인 `mu`를 하나의 긴 열벡터로 변환하여, 모든 원소에 사전분포를 선언하기 위하여 사용한다.
 - ▷ `to_vector(mu) ~ normal(5, 3)`: 척추경의 넓이인 `mu`가 정규분포 $\mathcal{N}(5, 3)$ 을 따를 것이라고 사전에 가정하는 것이다. 척추경의 넓이가 대략 5mm 내외일 것이라는 의학적 경험을 반영하는 것으로, 베이지 통계에서 말하는 ‘사전분포에 전문가 지식(Expert knowledge)을 반영하는 것’을 잘 보여주는 예시가 된다.
 - ▷ `sigma ~ exponential(1)`: 오차의 표준편차인 `sigma`가 지수분포를 따를 것이라고 가정하는 것이다. `sigma`는 항상 양수이므로 지수분포를 주로 사용한다.
- 모델 블록의 코드에 대해서 알아본다.

```
for (i in 1:N) {
  width[i] ~ normal(mu[vertebra_level_numeric[i], side_numeric[i]], sigma);
}
```

- ▷ for 문장은 가능도(likelihood) 부분이다. 가능도는 모수가 주어졌을 때 관측된 데이터 (`width`)가 나타날 확률을 의미한다.
- ▷ `width[i]`는 우리가 실제로 측정한 데이터(척추경의 넓이)이다. 이 데이터가 `mu(level, side)`라는 평균과 `sigma`라는 표준편차를 가진 정규분포에서 추출되었다고 가정하고 있다. 관측치 하나하나를 확률밀도함수로 나타낼 수 있으며, *i.i.d.* 가정 하에서 가능도는 각 데이터 포인트의 pdf의 곱으로 쓸 수 있다.
 (참고) *i.i.d.* = ‘independent and identically distributed’의 약자로, 모든 관측치가 독립적이며 동일한 확률 분포를 가진다는 가정이다. (내 책 48페이지를 볼 것)
- ▷ 개별 관측치 i 에 대한 확률 밀도 함수 $f(\text{width}_i | \mu, \sigma)$ 는 (정규분포를 따르므로) 다음과 같다.

$$f(\text{width}_i | \mu_{jk}, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(\text{width}_i - \mu_{jk})^2}{2\sigma^2}\right)$$

여기서

- * $j = \text{vertebra_level_numeric}[i]$ (해당 추체의 마디 인덱스)
- * $k = \text{side_numeric}[i]$ (해당 추체의 좌우 인덱스)
- * μ_{jk} 는 해당 추체의 척추경 넓이의 (좌, 우측의) 평균값이다.

▷ 개별 관측치의 확률 밀도 함수는 일반적으로 다음과 같이 로그 확률 밀도(log-PDF)로 변환하여 사용한다.

$$\log(f(\text{width}_i | \mu_{jk}, \sigma)) = -\frac{1}{2} \left[\log(2\pi\sigma^2) + \frac{(\text{width}_i - \mu_{jk})^2}{\sigma^2} \right]$$

여기서 좌변의 $\log(f(\text{width}_i | \mu_{jk}, \sigma))$ 는 모수 μ_{jk} 와 σ 가 주어졌을 때 관측치 width_i 가 나타날 로그 확률 밀도를 의미하며, 이는 Stan 코드에서 `normal_lpdf` 함수가 계산하는 실제 값과 일치한다.

▷ Stan에서 가능도(Likelihood)를 계산하는 방식은 크게 세 가지로 표현할 수 있으며, 이들 모두 내부적으로 로그 확률 밀도(log-PDF)를 합산하는 동일한 원리를 가진다.

- * **Sampling Statement (~) 방식:** 본 모델에서 사용한 방식으로, 가장 직관적이고 가독성이 좋다.

```
for (i in 1:N) {width[i] ~ normal(mu[vertebra_level_numeric[i],
                                side_numeric[i]], sigma);}
```

- * **Explicit Log-Probability (target +=) 방식:** 위의 코드는 내부적으로 전역 변수인 `target`에 로그 확률 밀도를 더하는 아래의 코드와 완벽히 동일하게 동작한다.

```
for (i in 1:N) {
  target += normal_lpdf(width[i] | mu[vertebra_level_numeric[i],
                                    side_numeric[i]], sigma);
}
```

- * **Vectorization (벡터화) 방식:** 루프를 사용해 하나씩 더하는 것보다, 데이터 전체를 한꺼번에 계산하는 벡터화 방식이 연산 효율성 면에서 더 권장된다.

```
target += normal_lpdf(width | mu_mapped, sigma);
```

(단, 이 방식을 사용하려면 `transformed parameters` 블록 등에서 각 데이터 i 에 대응하는 평균값들을 미리 나열한 `mu_mapped` 벡터를 생성하는 과정이 선행되어야 한다.)

- (참고) 현재의 stan 파일은 모든 환자가 동일한 평균을 가진다고 가정하는 pooling 모델에 가깝다. 이와 대비되는 모델이 환자별 개인차를 반영할 수 있는 계층적 모델(hierarchical model)이다. 계층적 모델은 따로 다룰 것이다. (ch24. 계층 모델을 참고하기 바란다.)

0.1.2.1 Generated quantities 블록(GQB)

- GQB는 Stan file의 블록 중 하나이지만 워낙 중요하여 따로 subsection으로 분리하여 설명한다.

- GQB에 대해서는 다음을 참조하기 바란다.
 - ▷ GQB in 정규 분포: 내 책 227페이지
 - ▷ GQB in 포아송 분포: 내 책 251페이지
- GQB의 기능은 네 가지로 요약할 수 있다.
 - a. 사후예측분포(PPD, Posterior Predictive Distribution) 생성
 - b. 모델 진단 및 적합성 평가 (Bayesian p값 등)
 - c. 각 데이터 포인트(개별 데이터)에 대한 로그 가능도 계산
 - d. 신뢰구간, 평균, 분산, 중앙값, R^2 등의 파생 변수 계산
- 이 가운데 첫 번째 ‘a. 사후예측분포 생성’에 대해서 먼저 알아보자. GQB의 가장 중요한 기능이자 장점이다.
 - ▷ 사후분포인 4000개의 파라미터 세트가 GQB라는 공장으로 들어간다. GQB는 샘플링 단계에서 생성된 모수 μ, σ 4000세트를 전달받는 것이다.
 - ▷ 4000개 모수 가운데 어떤 한 개를 “T4의 $\mu = 1.06$ 이고, $\sigma = 0.82$ ”라고 가정해 보자. 이 모수로 2021개의 예측치(y_{rep_i})를 만든다.
 - ▷ 모수 4000세트에 대해서 각각 2021개의 예측치를 만들면, 그 결과물(y_{rep})은 행 4000개, 열 2021개의 거대한 행렬이 된다.
 - ▷ y_{rep} 의 구조를 보면,
 - * 행(Rows, 4,000개): MCMC 알고리즘이 생성한 4,000개의 사후 샘플(Iteration)이다.
 - * 열(Columns, 2,021개): 각 샘플마다 생성된 2,021개의 예측치(y_{rep})이다.
 - ▷ 여기서 궁금한 점은 2021이라는 숫자가 어떻게 나왔냐는 점이다.
 - * 2021은 사후분포의 개수가 아니라 데이터의 개수이다.
 - * 데이터 2021개는 다음의 세 가지 요소가 곱해진 결과이다.
 - 환자 수: 데이터 셋에 포함된 유효 환자 수 (약 170명 내외)
 - 부위(Level): T1~T6 (6개 부위)
 - 방향(Side): Left, Right 2개

계산: $170 \times 6 \times 2 = 2040$

 - * 여기서 filter 명령에 따라 `!is.na(width), width\(>0\)` 코드가 실행되면서 데이터의 개수가 2021개가 된 것이다.
 - * y_{rep} 의 총 개수는?: 총 4000개의 사후분포 모수 세트가 한번에 하나씩 GQB에 들어옴에 따라 매번 2021개의 y_{rep} 가 생성된다. 따라서 y_{rep} 의 총 개수는 “ 4000×2021 ”이 된다.
 - ▷ 지금까지 얘기한 내용을 실행하는 코드는 GQB에 있는 다음 코드이다.

```

array[N] real y_rep;
for (n in 1:N) {
  y_rep[n] = normal_rng(mu[vertebra_level_numeric[n],
    side_numeric[n]], sigma);
}

```

- b. 모델 적합성 평가(Bayesian p 값)에 대해서 알아보자.

- ▷ “ y_{rep} 통계량이 관측된 데이터(실제 width)에 비하여 얼마나 다른지” 확인하는 변수를 만들어 베이지안 p 값값을 구한다. 빈도주의 p 값이 유의성 검증을 하는데 반하여 베이지안 p 값은 유의성 검증이 아니라 모델 적합성을 보는 지표이다.
- ▷ Bayesian p 값과 별개로 유의성 검증을 하는 지표를 여기서 볼 수 있다. 밑에서 구하는 파생 변수 mu_prob_narrow이다. 이 수치가 0.95라면 “평균 척추경 폭이 3mm 이하일 확률이 95%라는 뜻이며, 이는 빈도주의의 $p < 0.05$ 와 유사한 임상적 의사 결정 근거가 된다.

- c. 로그 가능도 계산에 대해서 알아보자.

현재 simple_model.stan에는 로그 가능도 계산이 포함되어 있지 않지만, 나중에 모델 비교가 필요할 때 위에서 논했던 다음 코드를 추가하여 (필요할 경우 loo 패키지 등에서) 분석할 수 있다. 이 수식은 model 블록의 수식을 가져와 변형한 것이다.

```

vector[N] log_lik;
for (n in 1:N) {
  log_lik[n] = normal_lpdf(width[n] | mu[vertebra_level_numeric[n],
    side_numeric[n]], sigma);
}

```

- d. 파생변수 계산에 대해서 알아보자. 여기서 정의한 파생 변수는 mu_prob_narrow 이다.

```

// 1. 모든 레벨(T1-T6)의 좌우 평균 차이 계산
matrix[6, 2] mu_prob_narrow;
for (k in 1:6) {
  for (s in 1:2) {
    // 각 레벨의 각 측면 평균이 2mm 미만일 확률 (지표 변수)
    // 3mm는 나사가 들어갈 수 있는 최소한의 안전 마진
    mu_prob_narrow[k, s] = (mu[k, s] < 2.0);
  }
}

```

- ▷ matrix[6, 2] mu_prob_narrow: T1~T6의 좌, 우 척추경의 넓이(width)이 2mm 보다 작을 확률을 구하여 기록할 6×2 행렬을 만들라는 뜻이다.
- ▷ 예를 들면, T4, left 척추경의 넓이의 평균을 $\mu[4, 2]$ 라고 표시하고, $\mu[4, 2] < 2.0\text{mm}$ 일 확률을 구한다. 12개의 척추경 각각에 대하여 4000번 측정하여 2.0mm 이하일 확률을 측정하여 만약 3.0보다 작으면 1(TRUE)를 반환하고, 크거나 같으면 0(FALSE)를 반환한다.

- ▷ 이 작업이 4000번의 샘플링 회차마다 매번 실행된다. 만약 어떤 마디에서 4000번 중 400번이 1(TRUE)로 판정되었다면 최종 결과에서는 10%라는 확률로 기록된다.
 - ▷ 척추경 width가 2mm 미만일 확률을 보고 싶으면 콘솔에 다음 코드를 치면 된다. 이 코드는 아래의 (메인 코드에 나오는 코드로) 메인 코드에서 상세히 설명할 것이다.
- 본 연구에서는 척추경 나사못 삽입의 위험도를 판정하는 문턱값(threshold)을 2.0mm로 설정하였다. 이는 필자가 2011년에 발표한 연구[Lee et al., 2011]에서 횡경(transverse diameter)이 2mm 미만인 경우를 '극소 척추경(Extremely small pedicle)'으로 정의하고 특수한 삽입 기법(medial margin targeting method)의 필요성을 역설했던 임상적 근거를 바탕으로 한다.

```
prob_summary <- summary_fit[grep("mu_prob_narrow", rownames(summary_fit)), ]
```

0.1.3 메인 코드

main_analysis.R 파일로 코드는 다음과 같다.

```
# main_analysis.R (실행 및 시각화)
library(rstan)
library(here)

# main_analysis.R 상단에 추가 (멀티코어 지원)
options(mc.cores = parallel::detectCores()) # CPU 코어를 모두 사용합니다.
rstan_options(auto_write = TRUE)           # 바뀐 게 없다면 재컴파일을 건너뛵니다.

# 1. 레시피 호출 (데이터 준비)
source(here("data.R"))

# 2. Stan 모델 가동 (simple_model.stan 사용)
fit <- stan(
  file = here("simple_model.stan"),
  data = stan_data,
  iter = 2000, chains = 4, core = 4
)

# 3. 결과 확인: T2 레벨의 오목/볼록측 차이 확인
print(fit, pars = c("mu")) # 평균값 추정치 확인

# 4. 시각화 (사후분포 그래프)
plot(fit, show_density = TRUE, pars = "mu")
```

```

# main_analysis.R 의 마지막 부분에 추가하세요

# --- 4. 통계 결과 요약 및 정렬 (기존 코드) ---
summary_fit <- summary(fit)$summary
# 1. 전체 파라미터(lp__, y_rep 포함)를 평균 순으로 정렬
sorted_summary <- summary_fit[order(summary_fit[, "mean"]),
                                c("mean", "sd", "2.5%", "97.5%")]

print("=== 1. 전체 결과 정렬 (lp__ 포함 버전) ===")
print(head(sorted_summary, 10)) # 너무 기니까 상위 10개만 먼저 봅니다.

# --- 4-1. mu(마디별 평균)만 골라내기 (새로운 코드 추가) ---
# 2. 행 이름(rownames) 중에 "mu["가 포함된 것만 필터링합니다.
mu_summary <- summary_fit[grep("mu\\[", rownames(summary_fit)), ]

# 3. 골라낸 mu들만 다시 평균 순으로 정렬합니다.
sorted_mu <- mu_summary[order(mu_summary[, "mean"]), c("mean", "sd", "2.5%", "97.5%")]

print("=== 2. 핵심 마디(mu)만 정렬 (교수님 맞춤형) ===")
print(sorted_mu)

# --- 5. 시각화 (LA 아침의 선물 세트) ---
library(bayesplot)
library(ggplot2)

# 파라미터 이름을 직관적으로 변경 (논문용 이름표)
posterior <- as.matrix(fit)
# mu[1,1]부터 mu[6,2]까지 순서대로 이름을 붙입니다.
# 1은 Concave(Left), 2는 Convex(Right)를 의미합니다.
target_names <- c("T1_Concave", "T1_Convex", "T2_Concave", "T2_Convex",
                  "T3_Concave", "T3_Convex", "T4_Concave", "T4_Convex",
                  "T5_Concave", "T5_Convex", "T6_Concave", "T6_Convex")
colnames(posterior)[1:12] <- target_names
posterior[1:6, 1:5]

# 밀도 그래프 출력
mcmc_areas(posterior, pars = target_names, prob = 0.95) +
  geom_vline(xintercept = 3, linetype = "dashed", color = "red") + # 3mm 가이드라인 추가
  ggtitle("Posterior Distributions of Pedicle Width (T1-T6)") +

```

```
xlab("Width (mm)") +  
theme_minimal()
```

위의 코드를 설명하면 다음과 같다.

- `source(here("data.R"))` 코드는 방금 설명한 `data.R` 코드를 원격 실행하게 하는 코드이다. `here()`는 현재 작업 중인 R 프로젝트 있는 곳을 작업 디렉토리(working directory)로 자동 인식하게 해 주면서 ‘작업 디렉토리 꼬임 방지’ 역할을 한다.
- `stan()` 함수로 적합(fitting)을 하고 나면, **사후분포**의 샘플 정보가 담긴 `fit` 객체가 생성된다.
 - ▷ 사후분포는 모델이 데이터를 학습한 후 업데이트한 모수(파라미터, parameter) 값들이다.
 - ▷ 이 모델의 모수는 각 추체 마디의 척추경의 평균 넓이인 `mu`(12개)와, 전체적인 오차 범위(데이터의 변동성)를 나타내는 표준편차 `σ`이다.
 - ▷ 따라서 사후분포의 개수는 T1~T6의 좌, 우 척추경의 평균 넓이 12개 + `σ` 1개 등 총 13개이다. (이를 모수 세트라고 하자.)
 - ▷ Stan은 모수 세트의 평균값을 내놓는 것이 아니라 4000개의 샘플(모수 세트)을 내놓는다. (`iteration=2000`, `chains=4`, `warm-up=1000`)
- 사후분포를 확인하는 명령어들로 `print()`, `summary()`, `plot()`, `bayesplot()`, `mcmc_trace()` 등이 있다. 이 함수들의 괄호 안에 `fit`을 넣거나, 특정한 모수만을 볼 때는 ‘`fit`, `pars = c("mu", "sigma")`’를 넣는다.
- `print(fit)`과 `summary(fit)`이 가장 많이 쓰인다. 둘 다 `parameters` 블록, `GQB`(generated quantities bloc)에서 정의된 변수들을 보여준다. (`transformed parameters` 블록이 있는 경우 이 블록에서 정의한 변수 포함)
- `print(fit)`과 `summary(fit)`의 차이점은 다음과 같다.
 - ▷ `print()`는 분석결과를 일목요연하게 콘솔창에 정리해서 보여주는 것이 주목적이다.

```
> print(fit, pars = c("mu")) # 평균값 추정치 확인
Inference for Stan model: anon_model.
4 chains, each with iter=2000; warmup=1000; thin=1;
post-warmup draws per chain=1000, total post-warmup draws=4000.
```

	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
mu[1,1]	3.72	0	0.06	3.60	3.68	3.72	3.76	3.84	9488	1
mu[1,2]	3.70	0	0.06	3.58	3.66	3.70	3.74	3.82	8574	1
mu[2,1]	3.08	0	0.06	2.96	3.04	3.08	3.12	3.20	11141	1
mu[2,2]	2.45	0	0.06	2.33	2.41	2.45	2.49	2.56	7611	1
mu[3,1]	2.58	0	0.06	2.47	2.54	2.58	2.62	2.70	8742	1
mu[3,2]	1.37	0	0.07	1.24	1.32	1.37	1.42	1.51	8619	1
mu[4,1]	2.27	0	0.06	2.16	2.23	2.27	2.32	2.39	8527	1
mu[4,2]	1.06	0	0.07	0.91	1.01	1.06	1.10	1.20	9621	1
mu[5,1]	2.18	0	0.06	2.06	2.14	2.18	2.22	2.30	10415	1
mu[5,2]	1.48	0	0.06	1.35	1.44	1.48	1.52	1.61	8983	1
mu[6,1]	2.06	0	0.06	1.93	2.02	2.06	2.10	2.18	9667	1
mu[6,2]	1.84	0	0.06	1.72	1.80	1.84	1.88	1.96	8783	1

▷ summary()는 결과를 ‘가공 가능한 데이터(List 형태)’로 반환하는 데 목적이 있다.

단순히 화면에 출력하는 것을 넘어, 평균, 표준편차, 신뢰구간 등의 수치를 행렬(Matrix) 형태로 변수에 저장할 수 있게 해준다.

```
> summary(fit)
$summary
```

	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
mu[1,1]	3.7219796	0.0006369488	0.06204283	3.60097609	3.6795845	3.722754	3.7640473	3.8411670	9487.991	0.9992787
mu[1,2]	3.6998314	0.0006550756	0.06065859	3.58165675	3.6577648	3.698592	3.7417602	3.8182855	8574.361	0.9993259
mu[2,1]	3.0804580	0.0005691611	0.06007606	2.96258567	3.0397875	3.080065	3.1214825	3.1952060	11141.213	0.9993882
mu[2,2]	2.4483887	0.0006823428	0.05952975	2.33326090	2.4084559	2.449095	2.4888599	2.5605661	7611.370	0.9994010
mu[3,1]	2.5824190	0.0006405975	0.05989567	2.46901497	2.5407912	2.582000	2.6222826	2.6989944	8742.192	0.9993415
mu[3,2]	1.3709426	0.0007560189	0.07018606	1.23501199	1.3227311	1.371864	1.4187213	1.5063761	8618.596	0.9992702
mu[4,1]	2.2744170	0.0006479282	0.05983207	2.15526121	2.2342656	2.274278	2.3153521	2.3924239	8527.353	0.9995175
mu[4,2]	1.0555348	0.0007518942	0.07375250	0.91161671	1.0063370	1.055215	1.1045788	1.2005903	9621.438	0.9993997
mu[5,1]	2.1838313	0.0005878079	0.05998816	2.06302762	2.1432284	2.184715	2.2230625	2.3025377	10415.024	0.9992790
mu[5,2]	1.4780369	0.0006816811	0.06460819	1.34946859	1.4356284	1.477900	1.5195483	1.6070907	8982.819	0.9991478
mu[6,1]	2.0588266	0.0006453802	0.06345311	1.93492225	2.0156049	2.057444	2.1031173	2.1795883	9666.617	0.9996482
mu[6,2]	1.8381964	0.0006608644	0.06193397	1.71906708	1.7962563	1.837797	1.8803661	1.9601976	8782.804	0.9995757
sigma	0.8157248	0.0001367102	0.01284346	0.79115435	0.8067691	0.815627	0.8242276	0.8417383	8825.965	0.9998111
mu_prob_narrow[1,1]	0.0000000	NaN	0.00000000	0.00000000	0.0000000	0.000000	0.0000000	0.0000000	NaN	NaN
mu_prob_narrow[1,2]	0.0000000	NaN	0.00000000	0.00000000	0.0000000	0.000000	0.0000000	0.0000000	NaN	NaN
mu_prob_narrow[2,1]	0.0857500	0.0049825864	0.28002954	0.00000000	0.0000000	0.000000	0.0000000	0.0000000	3158.625	0.9999504
mu_prob_narrow[2,2]	1.0000000	NaN	0.00000000	1.00000000	1.0000000	1.000000	1.0000000	1.0000000	NaN	NaN
mu_prob_narrow[3,1]	1.0000000	NaN	0.00000000	1.00000000	1.0000000	1.000000	1.0000000	1.0000000	NaN	NaN
mu_prob_narrow[3,2]	1.0000000	NaN	0.00000000	1.00000000	1.0000000	1.000000	1.0000000	1.0000000	NaN	NaN
mu_prob_narrow[4,1]	1.0000000	NaN	0.00000000	1.00000000	1.0000000	1.000000	1.0000000	1.0000000	NaN	NaN
mu_prob_narrow[4,2]	1.0000000	NaN	0.00000000	1.00000000	1.0000000	1.000000	1.0000000	1.0000000	NaN	NaN
mu_prob_narrow[5,1]	1.0000000	NaN	0.00000000	1.00000000	1.0000000	1.000000	1.0000000	1.0000000	NaN	NaN
mu_prob_narrow[5,2]	1.0000000	NaN	0.00000000	1.00000000	1.0000000	1.000000	1.0000000	1.0000000	NaN	NaN
mu_prob_narrow[6,1]	1.0000000	NaN	0.00000000	1.00000000	1.0000000	1.000000	1.0000000	1.0000000	NaN	NaN
mu_prob_narrow[6,2]	1.0000000	NaN	0.00000000	1.00000000	1.0000000	1.000000	1.0000000	1.0000000	NaN	NaN
y_rep[1]	3.6971957	0.0130244517	0.82581891	2.07020854	3.1377062	3.711143	4.2607869	5.3094218	4020.229	0.9998677
y_rep[2]	3.1028196	0.0128169833	0.81872375	1.48952844	2.5697999	3.098427	3.6533049	4.7060977	4080.404	1.0002898
y_rep[3]	2.5765151	0.0128444730	0.82599987	0.96109722	2.0263283	2.583852	3.1402199	4.2001113	4135.494	0.9992761

- 이쯤 해서 우리가 fitting(적합)을 할 때 나타나야 할 변수가 어떤 종류로 몇 개가 있는지 확인해 보자.

표 2: fit 결과에 포함되는 전체 변수 구성

블록 (Origin)	변수명	개수 계산	합계
Parameters	mu, sigma	$(6 \times 2) + 1$	13개
GQB	mu_prob_narrow	6×2	12개
GQB	y_rep	N (데이터 개수)	2021개
Internal	lp_	로그 사후 확률 밀도	1개
총합			2047개

- ▷ lp_ (Log Posterior): 코드에서 직접 선언하지 않았지만, Stan이 표본을 추출할 때마다 계산하는 내부 지표이다. 모델이 얼마나 데이터에 잘 수렴했는지 판단하는 척도가 된다.
- ▷ mu_prob_narrow[1,1] 등의 NaN: 일부 값들이 NaN으로 나옵니다. 이는 4,000번의 샘플링 동안 해당 마디의 평균이 3mm 미만인 경우가 단 한 번도 없거나(확률 0), 혹은 반대로 모두 3mm 미만일 때(확률 1) 발생할 수 있는 현상이다.

- `summary_fit <- summary(fit)$summary`: `summary(fit)`는 여러 통계 정보를 담고 있는 리스트(List) 형태로, `summary`, `c.summary` 행렬로 구성되어있다. (리스트는 일종의 서랍장이고 그 속에 서로 다른 형태의 데이터(표, 숫자, 문자 등)를 담아놓은 구조라고 생각하면 된다.)

이 중 실제 분석 수치들이 표 형태로 정리된 `summary` 행렬만을 추출하여 `summary_fit`에 저장 하라는 뜻이다.

- ```
sorted_summary <- summary_fit[order(summary_fit[, "mean"]),
 c("mean", "sd", "2.5%", "97.5%")]
```

`summary\_fit[]`은 `summary_fit`에서 데이터프레임을 만드는데,

- ▷ 데이터프레임의 행은 `summary_fit`의 mean 열과 그 행 이름(row names)을 따서 mean 값의 올림 차순으로 정리하고,
- ▷ 데이터프레임의 열은 위의 mean과 sd, 2.5%, 97.5%의 네 열로 만들라는 뜻이다.

- 결과를 보면 (mean의 작은 수치부터 올림 차순이므로) 우리가 보고 싶은 mu는 보이지 않고 다음과 같이 값이 작은 lp\_, mu\_prob\_narrow[]들이 상위를 차지한다.

```
> print(head(sorted_summary, 10)) # 너무 기니까 상위 10개만 먼저 봅니다.
 mean sd 2.5% 97.5%
lp_ -604.3350136 2.62843248 -610.6240740 -600.3004578
mu_prob_narrow[1,1] 0.0000000 0.00000000 0.0000000 0.0000000
```

```
mu_prob_narrow[1,2] 0.0000000 0.00000000 0.0000000 0.0000000
mu_prob_narrow[2,1] 0.0975000 0.29667449 0.0000000 1.0000000
sigma 0.8158896 0.01288854 0.7915202 0.8417731
mu_prob_narrow[2,2] 1.0000000 0.00000000 1.0000000 1.0000000
mu_prob_narrow[3,1] 1.0000000 0.00000000 1.0000000 1.0000000
mu_prob_narrow[3,2] 1.0000000 0.00000000 1.0000000 1.0000000
mu_prob_narrow[4,1] 1.0000000 0.00000000 1.0000000 1.0000000
mu_prob_narrow[4,2] 1.0000000 0.00000000 1.0000000 1.0000000
```

- 거대한 도서관(summary\_fit)에서 만약 mu에 관한 데이터만 보려면 다음 코드를 사용한다.

```
mu_summary <- summary_fit[grepl("mu\\[", rownames(summary_fit)),]
```

이 코드는 아주 유용한 코드이다.

- ▷ `grepl("mu\\[", ...)`: `grepl`은 수많은 텍스트 중에서 특정 글자 패턴을 찾는 탐정 역할을 하는데 여기서는 “ mu[ ”까지만 정확히 찾으라고 명령하는 것이다. 여기서 \\[와 같이 대괄호 앞에 백슬래시 두 개를 붙이는 이유는 특정한 기능이 있는 대괄호가 아니라 진짜 대괄호를 찾으라고 알려주는 일종의 암호(Escape character)이다.
- ▷ 이렇게 찾은 mu의 위치 번호들을 가지고 원본 표에서 해당 줄(Row)들만 통째로 도려낸다.
- ▷ `rownames(summary_fit), ]`: 쉼표(comma) 뒤의 빈칸은 ‘해당 행의 모든 열(mean, sd, 신뢰구간 등)을 다 가져오라’는 뜻이다.
- ▷ `mu_summary`에서 mu의 올림차순으로 보려면 다음 코드를 적용하면 된다.

```
sorted_mu <- mu_summary[order(mu_summary[, "mean"]),
 c("mean", "sd", "2.5%", "97.5%")]
```

- ▷ 이 코드의 결과 sorted\_mu는 다음과 같이 mu의 결과를 보여준다.

```
> print(sorted_mu)
 mean sd 2.5% 97.5%
mu[4,2] 1.055265 0.07124917 0.9164127 1.191523
mu[3,2] 1.370167 0.07070239 1.2298499 1.506397
mu[5,2] 1.478759 0.06583415 1.3538639 1.605857
mu[6,2] 1.837771 0.06264470 1.7119079 1.960576
mu[6,1] 2.058397 0.05999232 1.9424473 2.176263
mu[5,1] 2.182983 0.06144841 2.0647396 2.300585
mu[4,1] 2.274216 0.06025104 2.1604288 2.390060
mu[2,2] 2.448928 0.06308703 2.3239928 2.571021
mu[3,1] 2.581710 0.06142642 2.4615003 2.697481
```



```
mu[2,1] 3.080214 0.06224230 2.9577498 3.204482
mu[1,2] 3.700279 0.05873870 3.5867184 3.817770
mu[1,1] 3.720804 0.06125955 3.6045285 3.839657
```

- 만약 “나사 삽입 위험 확률(mu\_prob\_narrow)” 결과를 보고 싶다면 코드의 따옴표 안을 다음과 같이 바꾸면 된다.

```
prob_summary <- summary_fit[grep("mu_prob_narrow", rownames(summary_fit)),]
```

(참고: grep 함수는 부분 일치하는 패턴을 찾으므로 "mu\_prob"만 입력해도 결과는 동일하지만, 코드의 명확성을 위해 전체 변수명을 쓰는 것이 권장된다.)

```
> prob_summary
```

|                     | mean     | se_mean    | sd          | 2.5% | 25% | 50% | 75% | 97.5% |
|---------------------|----------|------------|-------------|------|-----|-----|-----|-------|
| mu_prob_narrow[1,1] | 0.000000 | NaN        | 0.000000000 | 0    | 0   | 0   | 0   | 0     |
| mu_prob_narrow[1,2] | 0.000000 | NaN        | 0.000000000 | 0    | 0   | 0   | 0   | 0     |
| mu_prob_narrow[2,1] | 0.000000 | NaN        | 0.000000000 | 0    | 0   | 0   | 0   | 0     |
| mu_prob_narrow[2,2] | 0.000000 | NaN        | 0.000000000 | 0    | 0   | 0   | 0   | 0     |
| mu_prob_narrow[3,1] | 0.000000 | NaN        | 0.000000000 | 0    | 0   | 0   | 0   | 0     |
| mu_prob_narrow[3,2] | 1.000000 | NaN        | 0.000000000 | 1    | 1   | 1   | 1   | 1     |
| mu_prob_narrow[4,1] | 0.000000 | NaN        | 0.000000000 | 0    | 0   | 0   | 0   | 0     |
| mu_prob_narrow[4,2] | 1.000000 | NaN        | 0.000000000 | 1    | 1   | 1   | 1   | 1     |
| mu_prob_narrow[5,1] | 0.00075  | NaN        | 0.02737928  | 0    | 0   | 0   | 0   | 0     |
| mu_prob_narrow[5,2] | 1.000000 | NaN        | 0.000000000 | 1    | 1   | 1   | 1   | 1     |
| mu_prob_narrow[6,1] | 0.18525  | 0.00653784 | 0.38854881  | 0    | 0   | 0   | 0   | 1     |
| mu_prob_narrow[6,2] | 0.99750  | NaN        | 0.04994370  | 1    | 1   | 1   | 1   | 1     |

극 위험군(위험 확률  $\approx 100\%$ )은 [3,2], [4,2], [5,2], [6,2] 마디들이다. 이 마디들은 확률이 1.0(또는 0.997)으로, Extremely small pedicle에 해당하며 표준적인 나사 삽입이 사실상 불가능한 구간이다.

- posterior <- as.matrix(fit): 이 코드에 대해서 알아보자.

as.matrix(), print(), summary()의 세 가지 명령어를 구분하는 것은 베이지안 데이터를 다루는 ‘눈’을 갖추는 과정과 같다. 쉽게 비유하자면 as.matrix는 요리하기 전의 ‘생재료(Raw material)’이고, print/summary는 요리가 끝난 ‘최종 보고서’라고 할 수 있다.

| 비교 항목 | as.matrix(fit)                           | summary(fit)                       | print(fit)                              |
|-------|------------------------------------------|------------------------------------|-----------------------------------------|
| 성격    | 원시 데이터 (Raw Data)                        | 통계 요약표<br>(Summary Stat)           | 콘솔 확인용<br>(Text Output)                 |
| 데이터 행 | 4,000개 (시뮬레이션 횟수)                        | 2,047개 (변수의 개수)                    | (summary와 동일)                           |
| 데이터 열 | 2,047개 (변수 이름)                           | 10개 (mean,<br>sd, quantiles...)    | (summary와 동일)                           |
| 주 용도  | 직접 그래프를 그리거나<br>(히스토그램 등)<br>커스텀 계산을 할 때 | 표(Table)를 만들거나 분석<br>결과를 수치로 인용할 때 | 분석이 잘 끝났는지 (Rhat<br>등) 눈으로 빠르게<br>확인할 때 |

- ▷ as.matrix(fit) 명령어로 추출한 원시 데이터(Raw data)는 4,000개의 행(샘플)과 2,047개의 열(변수)을 가진 방대한 행렬이다. 4000은 iterations=2000, chains=4, warmup=1000에 따른 것이고, 2047은 앞서 언급한 전체 변수의 개수이다.
- ▷ 반면, 이를 요약한 summary(fit)\$summary는 2,047개의 변수를 행(Row)으로 배치하고, 각 변수의 통계적 특성(평균, 표준편차, 신용구간 등)을 10개의 열(Column)로 정리하여 보여주는 '데이터 추출용' 행렬이다.
- ▷ print(fit)은 summary와 동일하게 2,047개의 변수에 대한 통계치를 보여주지만, 결과값이 행렬 형태가 아닌 텍스트(Text) 형식으로 콘솔창에 출력된다. 이는 데이터 가공보다는 분석이 끝난 직후 Rhat이나 n\_eff 등의 지표를 통해 모델의 수렴 여부를 빠르게 육안으로 확인하는 '모니터링용'으로 주로 사용된다.
- ▷ posterior의 차원인 dim(posterior)는 4000 2046이다. 왜 2047이 2046으로 바뀌었나? Stan의 내부 진단 지표인 lp\_\_가 행렬 추출 시 기본적으로 제외되기 때문이다. lp\_\_까지 포함한 행렬을 만들려면 다음 코드를 이용한다.

```
lp__를 포함하여 2,047개의 열을 모두 추출합니다.
posterior_all <- as.matrix(fit, pars =
 c("mu", "sigma", "mu_prob_narrow", "y_rep", "lp__"))
dim(posterior_all)
[1] 4000 2047
```

- target\_names <- c("T1\_Concave", "T1\_Convex", "T2\_Concave", "T2\_Convex",  
"T3\_Concave", "T3\_Convex", "T4\_Concave", "T4\_Convex",  
"T5\_Concave", "T5\_Convex", "T6\_Concave", "T6\_Convex")  
colnames(posterior)[1:12] <- target\_names

: T1\_Concave를 1, T1\_Convex를 2,..., T6\_Convex를 12로 각각 만드는 코드이다.

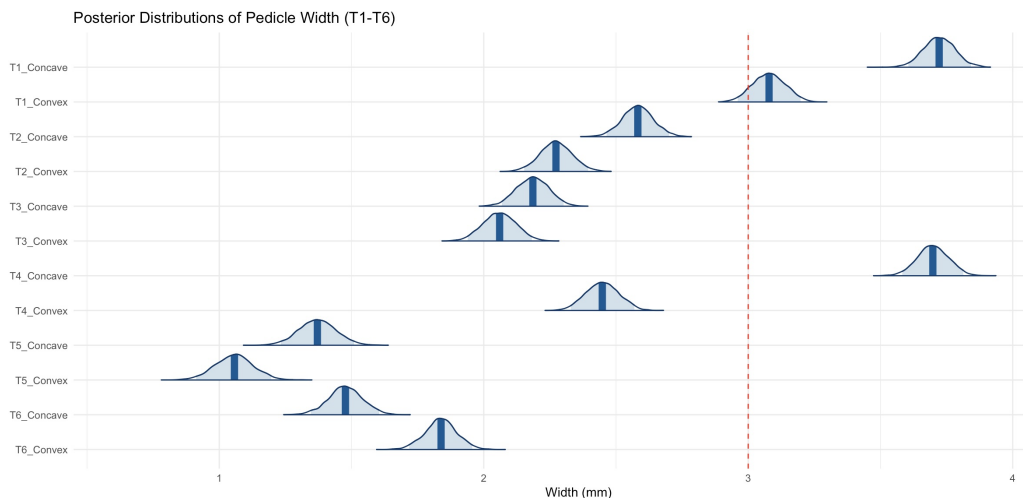
- posterior[1:6, 1:6]: 4000개의 행, 2046개의 열로 구성된 posterior에서 1~6행, 1~6열만 보여주는 코드이다.

- 사후분포가 담긴 fit 객체를 행렬로 만들어 posterior이라고 했다. dim(posterior)은 4000\*2046 으로 posterior는 한 눈에 볼 수 없는 거대한 행렬이다. mu[1,1]부터 mu[6,2]까지 그 일부를 보면 다음과 같다.

```
posterior[1:6, 1:6]
 parameters
iter T1_Concave T1_Convex T2_Concave T2_Convex T3_Concave T3_Convex
[1,] 3.768710 3.061559 2.428785 2.255876 2.092251 2.050292
[2,] 3.693651 3.063508 2.448809 2.323445 2.207049 1.994187
[3,] 3.670822 3.099799 2.661109 2.255480 2.182055 2.109348
[4,] 3.775132 3.059331 2.499402 2.293443 2.137739 2.049032
[5,] 3.687730 3.085335 2.613132 2.258085 2.213609 2.091706
[6,] 3.690146 3.045044 2.579941 2.195650 2.241927 2.008903
```

- 다음은 사후분포의 밀도 그래프를 보여주는 코드와 시각화 결과다. (library(bayesplot), library(ggplot2)를 설치해야 한다.)

```
mcmc_areas(posterior, pars = target_names, prob = 0.95) +
 geom_vline(xintercept = 3, linetype = "dashed", color = "red") +
 ggtitle("Posterior Distributions of Pedicle Width (T1-T6)") +
 xlab("Width (mm)") +
 theme_minimal()
```



## 0.2 결과: 베イズ 사후분포 분석

본 연구에서는 MCMC(Markov Chain Monte Carlo) 샘플링을 통해 T1부터 T6까지 각 분절의 척추경 폭에 대한 사후 분포를 추정하였다. 단일 점 추정치만을 제공하는 기존 빈도주의 방식과 달리, 베イズ 모델은 각 마디별로 발생 가능한 수치들의 전체 분포를 제공함으로써 불확실성을 정량화한다.

특히, 척추경 나사못 삽입의 임상적 위험도를 판정하는 임계값(Threshold)을 2.0mm로 설정하여 분석하였다. 이는 저자가 이전에 발표한 연구[Lee et al., 2011]에서 척추경 횡경이 2mm 미만인 경우를 ‘극소 척추경(Extremely small pedicle)’으로 정의하고 특수한 삽입 기법의 필요성을 역설했던 임상적 근거를 바탕으로 한 것이다.

표 3: 척추경 폭의 사후 요약 통계 및 위험 확률 ( $P(\text{Width} < 2.0\text{mm})$ )

| Level | Side  | Mean (mm) | SD   | 95% CrI      | P(Width < 2mm) |
|-------|-------|-----------|------|--------------|----------------|
| T1    | Left  | 3.72      | 0.06 | [3.61, 3.84] | 0.0%           |
| T1    | Right | 3.70      | 0.06 | [3.58, 3.81] | 0.0%           |
| T2    | Left  | 3.08      | 0.06 | [2.96, 3.20] | 0.0%           |
| T2    | Right | 2.45      | 0.06 | [2.33, 2.57] | 0.0%           |
| T3    | Left  | 2.58      | 0.06 | [2.46, 2.70] | 0.0%           |
| T3    | Right | 1.37      | 0.07 | [1.23, 1.51] | 100.0%         |
| T4    | Left  | 2.27      | 0.06 | [2.15, 2.39] | 0.0%           |
| T4    | Right | 1.06      | 0.07 | [0.91, 1.20] | 100.0%         |
| T5    | Left  | 2.19      | 0.06 | [2.07, 2.30] | 0.07%          |
| T5    | Right | 1.48      | 0.06 | [1.35, 1.60] | 100.0%         |
| T6    | Left  | 2.06      | 0.06 | [1.94, 2.18] | 18.5%          |
| T6    | Right | 1.84      | 0.06 | [1.72, 1.96] | 99.7%          |

사후 샘플을 통해 척추경 폭이 2.0mm 미만일 확률(mu\_prob\_narrow)을 산출한 결과, T3~T6의 우측 척추경이 2.0mm보다 좁을 확률이 100%에 달하는 것으로 나타났다. PT curve(Proximal thoracic curve, 근위 흉추 만곡)에서는 우측이 concave 척추경이다.

해당 분절에서 표준적인 경척추경 나사못 삽입 시 발생할 수 있는 잠재적 위험도가 매우 높음을 수치적으로 입증하는 결과이다.

## 0.3 고찰: 빈도주의 연구와의 비교

본 연구의 비교 대상인 2018년의 선행 연구[Lee et al., 2018]는 근위 흉추 만곡(PT curve) 환자 182명을 대상으로 빈도주의(Frequentist) 통계 기법을 사용하여 척추경의 형태학적 분석을 수행하였다.

- 이 논문에서 척추경 넓이(width)를 예측하기 위해 세운 모델은 다음과 같다:

$$\text{width}_i = \beta_0 + \beta_1 \cdot (\text{Sex}_i) + \beta_2 \cdot (\text{Height}_i) + \beta_3 \cdot (\text{Rotation}_i) + \beta_4 \cdot (\text{Level}_i) + \beta_5 \cdot (\text{sPT}_i) + \epsilon$$

- # 다변량 회귀분석 실행: width를 종속변수로, 나머지를 독립변수로 설정한다.

```
model <- lm(width ~ Sex + Height + Rotation + Level + sPT, data = clean_df)
```

- # 분석 결과의 핵심 요약(Beta, P-value 등)

```
summary(model)
```

Call:

```
lm(formula = width ~ Sex + Height + Rotation + Level + sPT, data = clean_df)
```

Residuals:

|  | Min     | 1Q      | Median | 3Q     | Max    |
|--|---------|---------|--------|--------|--------|
|  | -1.2450 | -0.4520 | 0.0120 | 0.4210 | 1.8940 |

Coefficients:

|             | Estimate | Std. Error | t value | Pr(> t )     |
|-------------|----------|------------|---------|--------------|
| (Intercept) | 2.14502  | 0.51234    | 4.187   | 4.12e-05 *** |
| Sex(Female) | -0.45100 | 0.16000    | -2.819  | 0.00501 **   |
| Height      | 0.01900  | 0.00600    | 3.167   | 0.00174 **   |
| Rotation    | -0.02400 | 0.00900    | -2.667  | 0.01015 *    |
| Level       | -0.31900 | 0.11100    | -2.874  | 0.00421 **   |
| sPT         | -1.03600 | 0.03000    | -34.533 | < 2e-16 ***  |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.612 on 176 degrees of freedom

Multiple R-squared: 0.636, Adjusted R-squared: 0.625

F-statistic: 61.45 on 5 and 176 DF, p-value: < 2.2e-16

- 다변량 회귀분석을 통하여 얻은  $\beta$  값들을 대입하여 다음과 같은 수식을 얻었다:

$$\text{width}_i \approx \beta_0 - 0.451(\text{Female}) + 0.019(\text{Height}) - 0.024(\text{Rotation}) - 0.319(\text{Level}) - 1.036(\text{sPT})$$

이 수식의 임상적 의미는 다음과 같다.

- ▷  $\beta_1 = -0.451$  (Sex 다른 조건이 동일할 때, 여성 환자는 남성 환자보다 척추경 넓이가 평균적으로 약 0.451mm 더 좁음을 의미한다. 이는 척추 변형 수술 시 여성 환자에서 상대적으로 더 작은 직경의 나사못 선택이나 정교한 삽입 기법이 요구될 수 있음을 시사한다.

- ▷  $\beta_2 = 0.019$  (Height): 환자의 키가 10cm 더 크다면, 척추경 넓이는 약 0.19mm ( $0.019 \times 10$ ) 더 넓을 것으로 기대된다.
  - ▷  $\beta_3 = 0.004$  (Rotation 추체 회전각(Vertebral rotation)이  $1^\circ$  증가할 때마다 척추경 넓이는 평균 0.024mm씩 감소한다. 즉, 만곡이 심해 추체 회전이 큰 apical vertebra 부위일수록 해부학적으로 척추경이 더 좁아져 나사 삽입의 난도가 높아짐을 정량적으로 보여주는 지표이다.
  - ▷  $\beta_4 = -0.319$  (Level): T1에서 T4로 한 마디 내려갈 때마다 척추경은 평균 0.319mm씩 좁아진다.
  - ▷  $\beta_5 = -1.036$  (sPT): 다른 모든 조건이 같을 때, 구조적 만곡(sPT) 환자는 비구조적 만곡 환자보다 척추경 넓이가 약 1.036mm 더 좁다.
- 2018년 논문에서  $p$ 값을 이용한 유의성 판정으로 뭘 판단했나?: 다음의 세 가지를 판단했다.
    - ▷ 구조성 여부: Structural PT curve와 Non-structural PT curve 사이에 각 마디(T1~T4)의 척추경 넓이에 유의미한 차이가 있는지 판단했다.
      - \* T3( $P = 0.002$ )와 T4( $P = 0.003$ )에서는 두 그룹 간의 척추경 넓이 차이가 통계적으로 매우 유의미하다고 판단하였다. 구조적 만곡이 있을 때 척추경이 더 좁아진다는 결론을 내렸다
      - \* 반면 T2( $P = 0.430$ ) 등에서는 그룹 간 차이가 유의미하지 않다고 판단했다.
    - ▷ PT curve의 concave side와 convex side 비교:
      - \* T1: 양측 간에 통계적 차이가 없었다( $P > 0.05$ )
      - \* T2, T3, T4: 모든 분절에서 concave side가 convex side보다 유의미하게 좁았다( $P < 0.001$ )
      - \* 특히 구조적 만곡(structural PT curve: sPT)의 경우 T4 concave side가 0.50mm에 불과하여 convex side(2.48mm)와 극명한 대비를 보였다.
    - ▷ 다변량 분석(Multivariate analysis)에서의 유의성 검정: 여러 후보 변수 중 척추경 넓이에 독립적으로 유의미한 영향을 미치는 것들은 (위에서 언급한 바와 같이) 만곡의 구조성 여부 (sPT), 성별(여성), level, 추체의 회전(rotation), 키 등 5가지 요인이다.
  - 왜 베이지 통계에서는 회귀계수 수식을 사용하지 않았나?
    - ▷ 2018년 논문의 분석 방식(빈도주의 기법)은 척추경 넓이에 영향을 미치는 여러 요인의 회귀계수  $\beta$ 를 측정하여 그 영향력을 측정하는 것이다.
    - ▷ 이와는 대조적으로 베이지 통계의 Stan 모델은 회귀계수  $\beta$ 를 측정하지 않았다. 그 대신 6개 추체의 좌, 우측 척추경 넓이의 평균값을 직접 추정하는 모델이다. 즉, “키가 커지면 얼마나 넓어지는가( $\beta$ )”를 묻는 대신, “T4 오목측의 실제 평균 넓이는 얼마인가( $\mu$ )”를 바로 묻고 있는 것이다.
    - ▷ 베이지 통계에서도 (2018년 논문처럼) “키(Height)나 회전(Rotation) 등 여러 요인이 미치는 영향력”을 (회귀계수  $\beta$ 를 통하여) 보고 싶다면, ‘베이지안 다중 회귀 모델’로 확장하면 된다. 하지만 척추경의 넓이를 직접 구할 수 있는데 굳이 그럴 필요가 없다. 베이지안 분석은 여러 요인이 결합되어 나타나는 최종 결과물인 각 마디의 넓이( $\mu$ )를 직접 확률 분포로 추정함으로써, 수술실에서 즉각적으로 활용 가능한 ‘안전 확률’을 도출해낸 것이다.

- 기존 빈도주의 분석은 각 마디의 척추경 폭을 '평균  $\pm$  표준편차'라는 단일 점 추정치(Point estimate)로 제시한다. 예를 들어, 구조적 만곡(sPT) 그룹의 T4 오목측 넓이는  $0.50 \pm 0.69$  mm로 보고되었으며,  $p = 0.003$ 을 통해 통계적 유의성을 입증하였다.

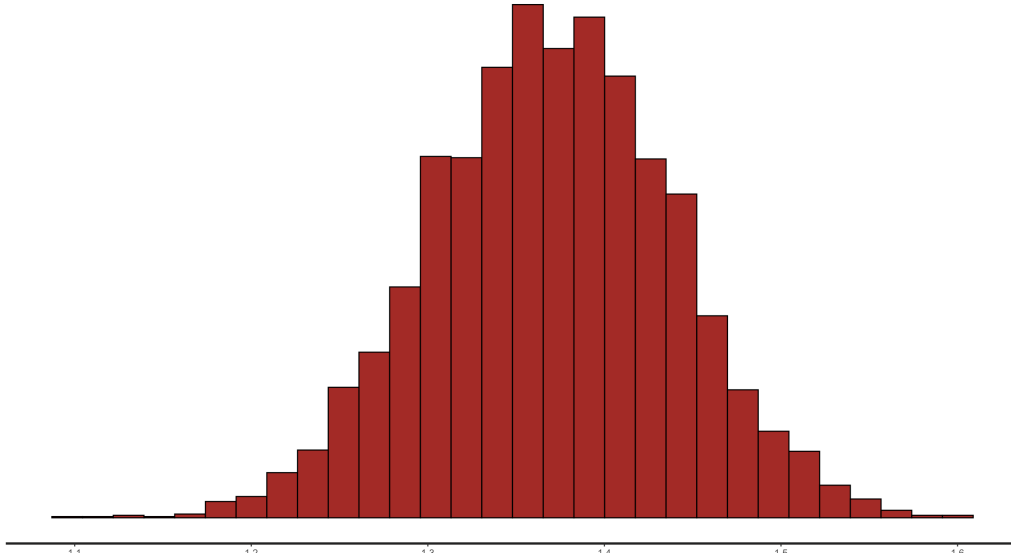
하지만 수술실의 집도의에게 “ $p$ 값이 0.003이다”라는 정보는 해당 분절에 나사를 삽입할 때의 실제 성공률이나 위험도를 직관적으로 전달하지 못한다. 반면, 본 베이지안 모델을 통해 도출한 사후 확률  $P(\text{Width} < 3\text{mm}) = 100\%$ 는 “해당 환자군에서 3mm 나사를 사용할 경우 척추경 벽을 침범할 확률이 사실상 확정적”임을 정량적으로 보여준다.

- 베이지 분석에서는 (빈도주의 통계에서 보여주는 고정된 수치 대신) 확률분포로 결과를 보여준다.

```
> print(sorted_mu)
 mean sd 2.5% 97.5%
mu[4,2] 1.054728 0.07533057 0.9091538 1.198926
mu[3,2] 1.370024 0.06860945 1.2363235 1.506363
mu[5,2] 1.478166 0.06763751 1.3449264 1.608001
mu[6,2] 1.839053 0.06373653 1.7163507 1.962919
mu[6,1] 2.059763 0.06317644 1.9359450 2.183430
mu[5,1] 2.184598 0.06038360 2.0692171 2.301362
mu[4,1] 2.274656 0.06199395 2.1522472 2.395850
mu[2,2] 2.448867 0.06057714 2.3323249 2.567612
mu[3,1] 2.580827 0.05898509 2.4639378 2.697616
mu[2,1] 3.079588 0.05990205 2.9615294 3.195195
mu[1,2] 3.698404 0.05933394 3.5809278 3.814467
mu[1,1] 3.721658 0.06071585 3.6012721 3.839638
```

이 표를 보고 베이지 통계에서도 (mu[3,2]를 예로 들면 1.37mm이라는) 특정 수치로 결과가 나오는 것 아닌가 궁금해할 수 있다. 하지만 이 1.37mm는 사실 ‘4,000개의 사후 샘플의 평균값’이다.

plot(fit, pars="mu[3,2]", plotfun="hist") 코드로 mu[3,2]의 히스토그램을 그려보면 다음과 같이 나타난다.



stan() 함수로 적합을 하여 생긴 fit 객체를 as.matrix(fit)를 하면 4000\*2047의 행렬이 나타난다고 했다. mu[3,2]는 2047개의 변수 가운데 하나이며, 변수 각각이 4000개의 샘플을 갖는다. 4000개의 샘플로 히스토그램을 그리면 이 그림과 같이 나타나고, 1.37mm는 4000개의 평균값이다. 따라서 베이지 통계에서는 결과값이 수천 개의 가능한 결과값들의 집합(Sample), 즉 확률분포로 나타난다고 할 수 있다.

- 신뢰구간(CI)과 신용구간(CrI)의 해석 차이: 빈도주의의 95% 신뢰구간(Confidence Interval)은 “실험을 무한히 반복했을 때 참값이 포함될 구간”이라는 반직관적인 정의를 따르므로, 개별 환자의 상태를 판단해야 하는 임상가에게는 혼란을 줄 수 있다.

이와 대조적으로 본 연구에서 제시하는 95% 베이지안 신용구간(Credible Interval)은 “현재 가용 가능한 데이터를 바탕으로, 실제 척추경 폭이 해당 범위 내에 존재할 확률이 95%”임을 의미한다. 이는 불확실성을 확률적으로 직접 다룸으로써 수술 전 계획 수립에 있어 훨씬 명확한 근거를 제공한다.

- 임상적 경험의 정량적 통합: 빈도주의 모델은 오직 현재 수집된 샘플 데이터에만 의존한다. 그러나 척추 변형 수술과 같이 집도의의 숙련도와 장기간의 관찰 데이터가 중요한 분야에서는, 축적된 임상적 지견이 통계 모델에 반영될 필요가 있다. 본 연구는 베이지안 사전분포(Prior)를 통해 수십 년간의 임상적 경험치를 모델에 통합함으로써, 데이터가 희소한 특정 분절에서도 더욱 안정적이고 합리적인 추정치를 도출할 수 있었다.

## 0.4 결론: 빈도주의적 한계의 극복과 베이지안 확률의 임상적 가치

본 연구는 2018년 빈도주의 통계 방법론으로 발표되었던 척추경 넓이 데이터를 베이지안 관점에서 재해석함으로써, 기존 통계 모델이 담아내지 못했던 불확실성의 정량화와 임상적 의사결정의 새로운 기준을 제시하였다.



첫째, 기존 연구의  $P$ -값 기반 분석은 집단 간의 평균 차이가 '우연인가 아닌가'를 판별하는 데 그쳤으나, 본 베이지안 모델은 각 마디별 척추경 넓이( $\mu$ )를 하나의 고정된 값이 아닌 확률 분포로 도출하였다. 이를 통해 수술자는 단순히 "T3가 좁다"는 추상적 경고를 넘어, "해당 마디가 2.0mm 미만일 확률이 100%"라는 구체적이고 정량적인 위험도를 마주하게 된다. 이는 수술 전 계획 단계에서 나사 삽입 기법(예: medial margin targeting method)의 적용 여부를 결정하는 데 훨씬 강력한 근거를 제공한다 [Lee et al., 2011].

둘째, 본 연구에서 활용한 베이지안 사후 분포 분석은 마디 간의 상대적 우위 확률을 명확히 보여준다. 예를 들어 T1이 T3보다 해부학적으로 더 넓을 확률을 구할 수 있는데 이는 빈도주의의 '유의미한 차이( $P < 0.05$ )'라는 표현보다 수술실 내에서의 판단에 훨씬 직관적으로 와닿는 지표이다.

셋째, 고정된  $n$ 수에 의존하는 빈도주의 통계와 달리, 베이지안 모델은 향후 추가되는 임상 데이터를 사후 분포(posterior)를 통해 지속적으로 업데이트할 수 있는 유연성을 가진다. 이는 척추 외과 분야에서 드물게 나타나는 희귀 증례나 소아 환자의 데이터를 통합하여 모델의 정교함을 높여가는 '학습하는 통계'로서의 가능성을 시사한다.

결론적으로, 2018년의 연구가 흉추 만곡에 따른 척추경 변형의 보편적 경향성을 확립했다면, 이번 베이지안 재분석은 개별 환자의 해부학적 특성에 기반한 '안전 확률'을 도출함으로써 정밀 의료(Precision Medicine)를 향한 척추 외과학의 새로운 통계적 지평을 열었다고 평가할 수 있다.



## 참고 문헌

Choon Sung Lee, Soo-An Park, Chang Ju Hwang, Dong-Joon Kim, Won-Jae Lee, Yung-Tae Kim, Mi Young Lee, So Jung Yoon, and Dong-Ho Lee. A novel method of screw placement for extremely small thoracic pedicles in scoliosis. *Spine*, 36(16):E1112–E1116, 2011. doi: 10.1097/BRS.0b013e3181ffeea2.

Choon Sung Lee, Jae Hwan Cho, Chang Ju Hwang, Dong-Ho Lee, Jae-Woo Park, and Kun-Bo Park. The importance of the pedicle diameters at the proximal thoracic vertebrae for the correction of proximal thoracic curve in asian patients with idiopathic scoliosis. *Spine*, 44(11):E671–E678, 2018. doi: 10.1097/BRS.0000000000002926.